Стенд: Ubuntu 24.04, ядро 6.8, nginx 1.24.0, dnsmasq 2.91, ipvsadm 1.31, tshark 4.2.2.
Команды: `run.sh`. Полный вывод: `output.log`. Дампы: `captures/`.

Linux: `sudo modprobe ip_vs ip_vs_rr dummy && sudo bash run.sh | tee output.log`
Docker: `docker run --rm --privileged -v "$PWD":/hw ubuntu:24.04 bash /hw/run.sh | tee output.log`

1. Пул серверов

```
mkdir -p ~/backend1 ~/backend2
echo '<h1>Response from Backend Server 1</h1>' > ~/backend1/index.html
echo '<h2>*** Response from Backend Server 2 ***</h2>' > ~/backend2/index.html
cd ~/backend1 && python3 -m http.server 8081 &
cd ~/backend2 && python3 -m http.server 8082 &
```

```
$ ss -ltnp | grep -E ':808[12]'
LISTEN 0  5  0.0.0.0:8082  0.0.0.0:*  users:(("python3",pid=3588,fd=3))
LISTEN 0  5  0.0.0.0:8081  0.0.0.0:*  users:(("python3",pid=3586,fd=3))
$ curl -s http://127.0.0.1:8081
<h1>Response from Backend Server 1</h1>
$ curl -s http://127.0.0.1:8082
<h2>*** Response from Backend Server 2 ***</h2>
```

Оба сервера работают на 8081 и 8082.

2. DNS-балансировка (dnsmasq)

`dnsmasq.conf`:

```
listen-address=127.0.0.1
bind-interfaces
no-resolv
no-hosts
host-record=my-awesome-highload-app.local,127.0.0.1
host-record=my-awesome-highload-app.local,127.0.0.2
```

```
$ dig @127.0.0.1 my-awesome-highload-app.local
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1
;; ANSWER SECTION:
my-awesome-highload-app.local. 0 IN A 127.0.0.1
my-awesome-highload-app.local. 0 IN A 127.0.0.2
```

4 повторных запроса `dig @127.0.0.1 my-awesome-highload-app.local +short`:

```
127.0.0.2 127.0.0.1
127.0.0.1 127.0.0.2
127.0.0.2 127.0.0.1
127.0.0.1 127.0.0.2
```

Отказ `backend2`:

```
$ pkill -f 'http.server 8082'
$ curl http://127.0.0.1:8082
curl: (7) Failed to connect to 127.0.0.1 port 8082 after 0 ms: Couldn't connect to server
$ dig @127.0.0.1 my-awesome-highload-app.local +noall +answer
my-awesome-highload-app.local. 0 IN A 127.0.0.2
my-awesome-highload-app.local. 0 IN A 127.0.0.1
```

Показатель	Значение
A-записей в ответе	2
Смена порядка	на каждый запрос (раунд-робин)
TTL	0 (dnsmasq по умолчанию для локальных записей)
A-записей после отказа `backend2`	2 (не изменилось)

- DNS не проверяет, живы ли бэкенды. После отказа он продолжает отдавать `127.0.0.2`, причём первым примерно в 50% ответов.
- Клиенты, которые берут первый адрес, примерно в половине случаев получают ошибку подключения.
- Запись нужно удалять вручную или через внешний хелсчек. После удаления клиенты ещё держат её в кэше до истечения TTL.

3. L4 (IPVS)

```
ip link add dummy1 type dummy
ip addr add 192.168.100.1/32 dev dummy1
ip link set dummy1 up
ipvsadm -A -t 192.168.100.1:80 -s rr
ipvsadm -a -t 192.168.100.1:80 -r 127.0.0.1:8081 -m
ipvsadm -a -t 192.168.100.1:80 -r 127.0.0.1:8082 -m
for i in $(seq 6); do curl -s http://192.168.100.1; done
```

```
$ ip -br addr show dummy1
dummy1  UNKNOWN  192.168.100.1/32

curl ×6: Server 2, Server 1, Server 2, Server 1, Server 2, Server 1

$ ipvsadm -Ln
TCP  192.168.100.1:80 rr
  -> 127.0.0.1:8081   Masq  1  0  3
  -> 127.0.0.1:8082   Masq  1  0  3

$ ipvsadm -Ln --stats
Prot LocalAddress:Port     Conns  InPkts  OutPkts  InBytes  OutBytes
TCP  192.168.100.1:80          6      39       34     2532      3190
  -> 127.0.0.1:8081            3      21       16     1344      1531
  -> 127.0.0.1:8082            3      18       18     1188      1659
```

Реальный сервер	`Conns`	`InActConn`	Доля
127.0.0.1:8081	3	3	50%
127.0.0.1:8082	3	3	50%

- Раунд-робин работает: 6 соединений разошлись 3 и 3, ответы чередуются.
- `-m` (NAT) обязателен, потому что порт меняется с 80 на 8081 и 8082.
- `InActConn` показывает закрытые соединения в состоянии `TIME_WAIT`.

4. L7 (NGINX, основной + резервный)

`nginx-highload.conf`, сайт по умолчанию удалён:

```
upstream highload_backend {
    zone highload_backend 64k;
    server 127.0.0.1:8081 max_fails=7 fail_timeout=30s;
    server 127.0.0.1:8082 backup;
}
server {
    listen 127.0.0.1:8888;
    location / {
        proxy_pass http://highload_backend;
        proxy_set_header X-high-load-test 123;
    }
}
```

```
$ ss -ltnp | grep nginx
LISTEN 0  511  127.0.0.1:8888  0.0.0.0:*  users:(("nginx",...))
```

tshark, один запрос `curl http://127.0.0.1:8888/`:

```
$ tshark -r normal.pcap -d tcp.port==8888,http -d tcp.port==8081,http -Y http -O http
 4  curl → nginx:8888    GET / HTTP/1.1      Host: 127.0.0.1:8888
 9  nginx → :8081        GET / HTTP/1.0      X-high-load-test: 123
                                             Host: highload_backend
                                             Connection: close
13  :8081 → nginx        HTTP/1.0 200 OK     Content-Length: 40  <h1>Response from Backend Server 1</h1>
18  nginx → curl         HTTP/1.1 200 OK     Content-Length: 40
```

Отказ основного сервера: `pkill -f 'http.server 8081'`, затем 10 запросов.

Лог доступа (`$upstream_addr`, `$upstream_status`):

```
 1-7   127.0.0.1:8081, 127.0.0.1:8082   502, 200
 8-10  127.0.0.1:8082                   200
```

Лог ошибок:

```
[error] connect() failed (111: Connection refused) ... upstream: "http://127.0.0.1:8081/"   ×7
[warn]  upstream server temporarily disabled ... upstream: "http://127.0.0.1:8081/"
```

```
$ tshark -r failover.pcap -Y 'tcp.dstport==8081 && tcp.flags.syn==1' | wc -l
7
```

Показатель	Значение
Ответов клиенту 200 / ошибок	10 / 0
Попыток подключения к 8081 (SYN)	7
`connect() failed` в логе ошибок	7
Первый запрос сразу на резервный	8-й
Воркеров nginx, обработавших неудачи	4 (общий счётчик через `zone`)

- Резервный сервер включается после 7 неудач подряд (`max_fails=7`).
- Клиент ошибок не видит: `proxy_next_upstream` повторяет каждый неудачный запрос на резервном сервере.
- Заголовок `X-high-load-test: 123` добавляется только в запрос nginx → бэкенд.
- Без `zone` у каждого воркера был бы свой счётчик, и переключение произошло бы только после 7 неудач на каждом.